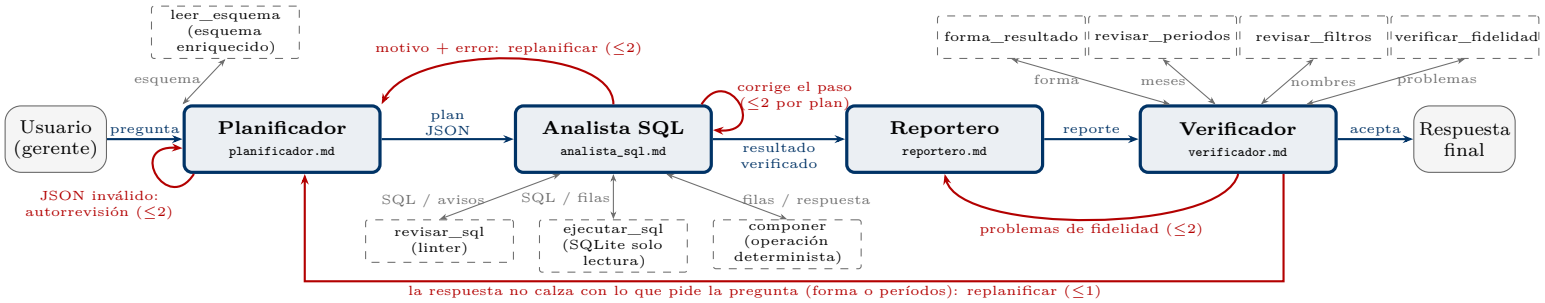
SQL4Business: Deliverable 3 — sistema de agentes

Generative Artificial Intelligence (580694), Universidad de Concepción
Aredhel Jiménez, Bryan Riquelme, Guido Salazar, Vicente Soñez

Código: github.com/aredhel-jmze/Asistente-SQL-4-Business | Video: [PENDIENTE: enlace del video]



Paradas: el Verificador acepta sin discrepancias o con los presupuestos agotados (si un plan nuevo falla, entrega la respuesta anterior; sin ella, no hay respuesta). Azul: agentes (mismo modelo, prompt propio); gris: herramientas deterministas; rojo: ciclos y su presupuesto.

Tarea, continuidad y modelo. La misma tarea, falla y criterio de salida correcta de los entregables anteriores. Las piezas deterministas del D2 pasan a ser herramientas de los agentes y el código del D2 no cambia. Los cuatro agentes usan el mismo modelo, `Qwen/Qwen2.5-Coder-3B-Instruct` (revisión `488639f`, NF4 de 4 bits, decodificación greedy, Colab T4), con un prompt propio por agente en `prompts/`.

Agentes y división del trabajo. El **Planificador** recibe la pregunta y el esquema y devuelve un plan JSON (pasos SQL y operación); si su JSON no es válido, revisa su propia salida. El **Analista SQL** ejecuta el plan con `revisar_sql`, `ejecutar_sql` y `componer`; solo llama al modelo cuando una herramienta falla, y entonces decide entre corregir el paso o devolver el plan con el motivo. El **Reportero** redacta la respuesta desde el resultado verificado. El **Verificador** lee solo la pregunta, declara qué tipo de respuesta pide y qué períodos nombra, y sus herramientas lo comparan con lo calculado: si no calza, devuelve el caso al Planificador; si el reporte no es fiel, al Reportero; si no, acepta. Se divide así porque son cuatro competencias distintas (entender la pregunta, interactuar con la base, redactar y juzgar) y porque quien juzga no debe ser quien produjo el trabajo.

Técnicas de prompting. Clasificadas según la taxonomía de Schulhoff et al. (2024). Todos los agentes usan *role prompting* y salida estructurada. *Planificador:* (1) **few-shot**, tres ejemplos resueltos sobre la misma base que no pertenecen a ningún conjunto de evaluación, contra la falla de devolver listados en vez del primero o de usar `strftime('%Q')`; (2) **descomposición** (*plan-and-solve*): antes del SQL declara qué debe contener la respuesta y cada período como rango de fechas, contra la falla de responder con una sola consulta una pregunta de varios valores. *Analista:* (3) **autocorrección guiada por herramientas** (*self-debugging*): el error de SQLite, del linter o de la composición vuelve al modelo, contra los errores de fechas y sintaxis (q09, q10 del D2). *Verificador:* (4) **verificación con lista de chequeo** (*self-verification*) contrastada con herramientas (forma del resultado, pasos descartados, meses y su orden, nombres de la base citados en la pregunta), contra respuestas válidas que no responden la pregunta (q03, q11, q09-p y q14-p del D2). *Reportero:* (5) **refinamiento con retroalimentación** (*self-refine*), contra cifras no respaldadas en el reporte (q13). Cada técnica se puede apagar por separado; la tabla mide el sistema sin cada una. Un test comprueba que los planes que fallaron en el D2 vuelven al agente que corresponde.

Evaluación. Las 15 preguntas del D1 (10 puntuales, 5 combinadas; in-sample) y 12 paráfrasis (5/7; externas). Mismas entradas y criterio para todas las configuraciones: *sql* (criterio del D1), *answer* (respuesta correcta), *e2e* (respuesta correcta y reporte fiel); *Llamadas*: promedio por pregunta. La primera corrida (v1) mostró fallos nuevos; v2 los corrige (ver desviaciones) y la tabla incluye ambas.

Resultados. [PENDIENTE: completar con la tabla medida: agentes frente a D2 y baseline en combinadas y totales, y qué técnica pesa más en la ablación].

Caso de falla. [PENDIENTE: pregunta en la que el sistema completo falla, con su traza en `results/deliverable3/trazas/` y la explicación de por qué].

Límites. El Verificador depende de que un modelo de 3B clasifique bien la pregunta; los contrastes trabajan a nivel de mes y de nombres, y un predicado equivocado con un reporte fiel a su resultado sigue sin detectarse. Cuesta más llamadas que el D2. Los conjuntos son chicos y sintéticos.

Desviaciones declaradas. Nuevo linter `revisar_sql` (del caso de falla del D2) y una regla de fidelidad compartida con el D2 (cambio porcentual repetido) que no altera sus veredictos. v2, tras la corrida v1: el Reportero recibe la indicación de cambio porcentual solo cuando aplica (en v1 la imitaba e inventaba cifras), el Verificador suma `revisar_filtros`, orden de períodos y pasos descartados, y el Analista puede invertir dos pasos. Como v2 se ajustó mirando fallos de ambos conjuntos, las paráfrasis ya no son estrictamente externas.

Reproducción. `notebooks/deliverable3_demo.ipynb` (Colab T4): demo (`run_agents.py`) y evaluación (`evaluate_d3.py`), con trazas en `results/`.

	Set oficial (15)			Paráfrasis (12)			
Configuración	sql	answer	e2e	sql	answer	e2e	**Llamadas**
Prompt directo (baseline D1)	7/15	8/15	–	3/12	5/12	–	1.0
Solución D2 (pipeline)	11/15	11/15	10/15	8/12	10/12	10/12	2.1
Agentes v1 (primera corrida)	11/15	11/15	9/15	9/12	11/12	9/12	3.8
Sistema de agentes (D3)	?	?	?	?	?	?	?
sin ejemplos (few-shot)	?	?	?	?	?	?	?
sin descomposición explícita	?	?	?	?	?	?	?
sin autocorrección del Analista	?	?	?	?	?	?	?
sin verificación crítica	?	?	?	?	?	?	?
sin refinamiento del reporte	?	?	?	?	?	?	?